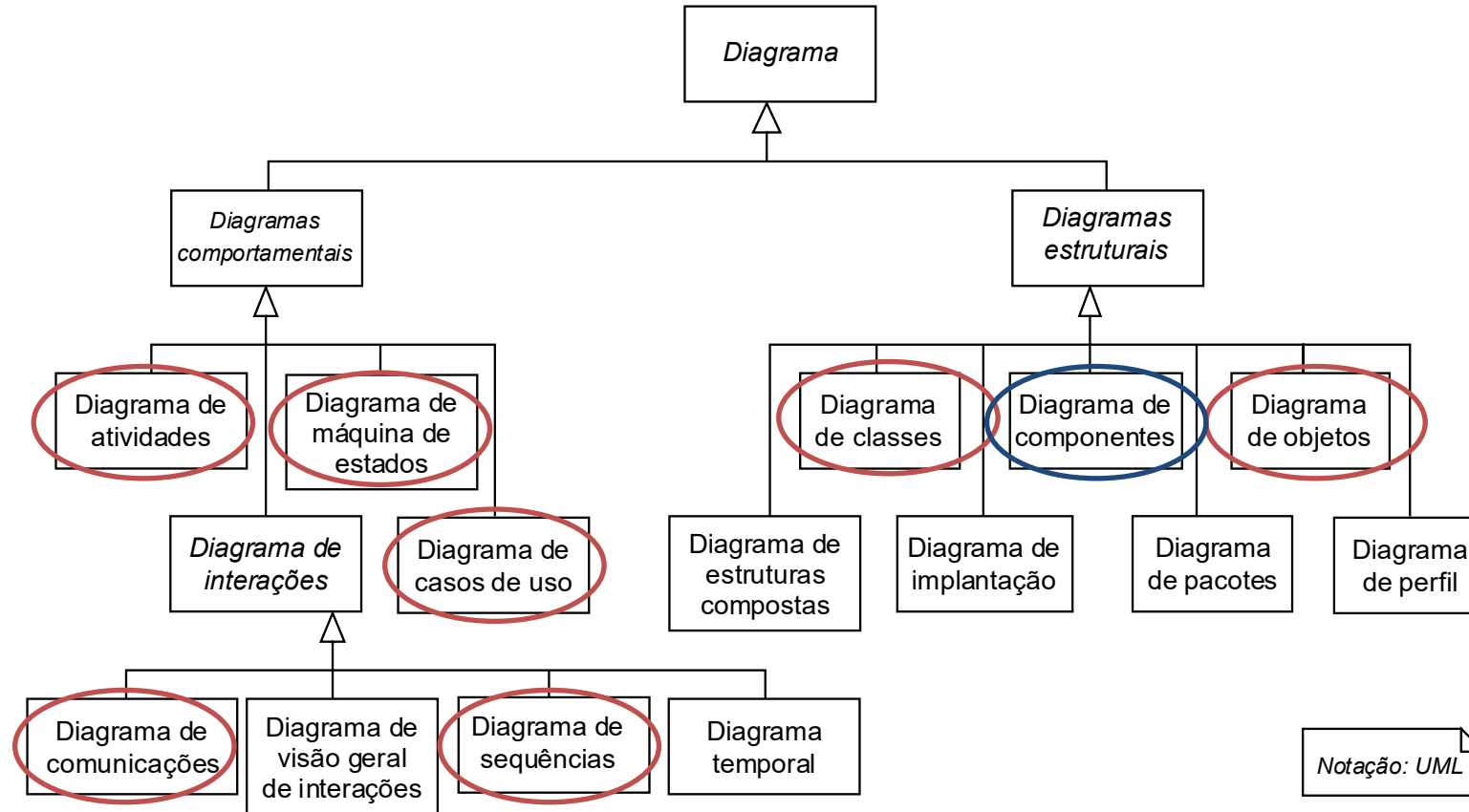




AMS II

Diagrama de Componentes

Diagramas UML



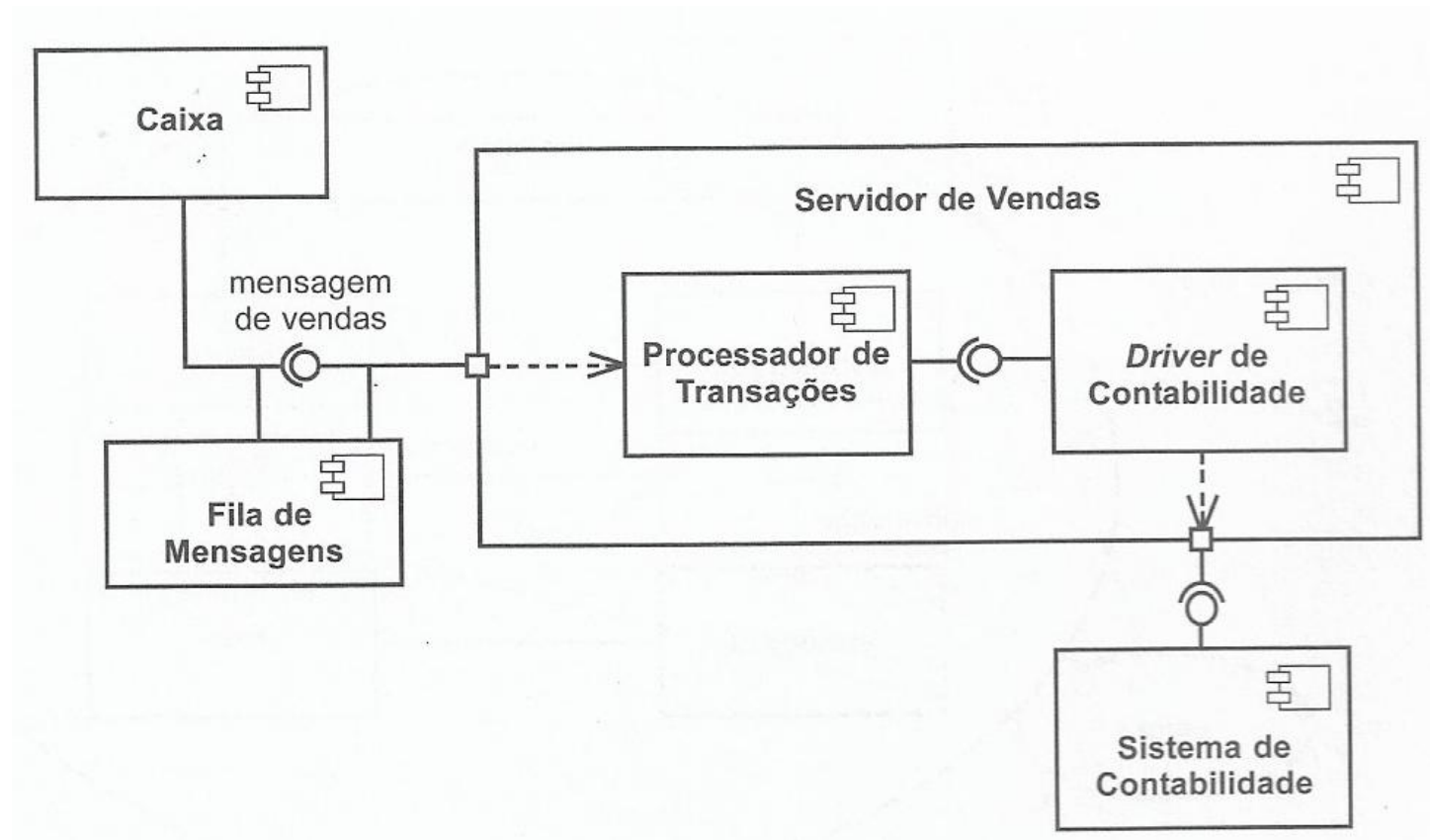
Notação: UML

Diagrama de Componentes

- O Diagrama de Componentes é um dos diagramas estruturais da UML.
- Ele representa **como o sistema é dividido em partes físicas ou lógicas reutilizáveis**, chamadas **componentes**, e **como essas partes se conectam** para formar o sistema completo.
- Esse diagrama mostra a **organização e as dependências** entre componentes de software, como **módulos, bibliotecas, serviços, APIs ou pacotes de código**.



Exemplo: Diagrama de Componentes



Finalidade

- O Diagrama de Componentes tem como principal objetivo **visualizar a arquitetura física ou de implementação** do sistema, facilitando a **reutilização e manutenção de código**.
- É utilizado para:
 - Mostrar **como os módulos estão estruturados e interligados**;
 - Identificar **dependências** entre partes do sistema;
 - Planejar a **implementação de subsistemas e bibliotecas**;
 - Auxiliar na **implantação**, em conjunto com o **Diagrama de Implantação**;
 - **Documentar sistemas grandes** de forma **modular e compreensível**.



Importância

- O diagrama é essencial no **gerenciamento de configuração e mudanças**, pois permite:
 - Modelar **baselines** (conjuntos estáveis de componentes);
 - Identificar **quais módulos ou arquivos pertencem** a uma versão específica do software;
 - Facilitar o **entendimento da estrutura geral** do sistema;
 - Melhorar a **comunicação** entre equipe técnica e partes interessadas;
 - Apoiar o **planejamento de integração e desenvolvimento** dos componentes.



Benefícios Práticos

- **Facilita a manutenção e atualização** do software;
- **Auxilia na análise de impacto** de mudanças;
- **Identifica problemas de projeto** e oportunidades de melhoria;
- **Complementa outros diagramas**, como o **Diagrama de Pacotes**, permitindo visualizar:
 - Os **componentes contidos** em cada módulo ou camada;
 - A **comunicação entre componentes** dentro e entre divisões da arquitetura.



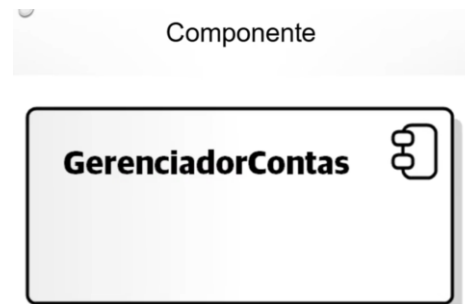
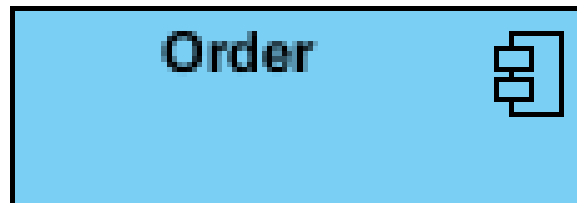
Relação com Outros Diagramas

- **Casos de Uso:** Define as funcionalidades que serão implementadas nos componentes.
- **Classe:** Mostra a estrutura interna de um componente.
- **Implantação:** Mostra onde os componentes serão executados fisicamente.



Componente

- Um **componente** é uma **unidade autônoma** dentro de um sistema ou subsistema.
- Ele é responsável por **implementar um conjunto de funcionalidades** específicas.



Componente

- Podem representar:
 - Componente **lógico**: implementa uma parte funcional do sistema
 - Componente de **negócio**: relacionado a processos ou regras de negócio
 - Componente **físico**: elementos concretos de implementação
- Características:
 - **Encapsulado**: seus detalhes internos não são acessíveis diretamente;
 - **Autônomo**: pode ser desenvolvido e testado de forma independente;
 - **Possui interfaces fornecidas e requeridas**;
 - **Transparente**: apenas suas interfaces estão visíveis.



Componente

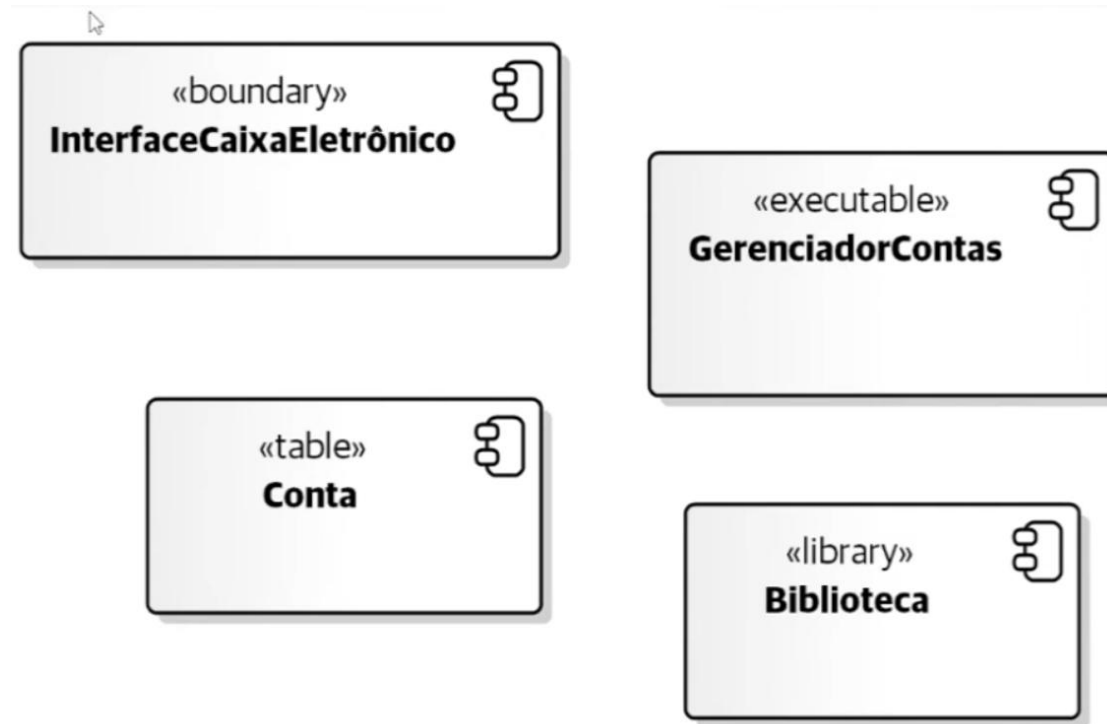
- Os **estereótipos** em UML permitem indicar **características específicas** de um componente.

«executable»	Arquivo executável (ex.: .exe)
«library»	Biblioteca reutilizável (ex.: .dll, .jar, .so)
«table»	Tabela de banco de dados
«boundary»	Interface de comunicação com o usuário ou outro sistema
«document»	Documento associado (ex.: relatório, manual)
«file»	Arquivo físico no sistema



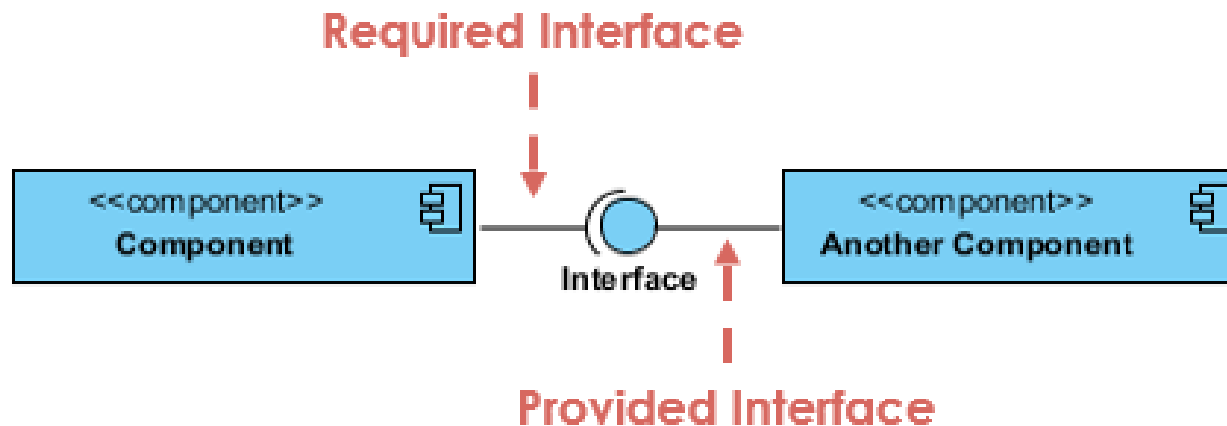
Componente

- Os **estereótipos** em UML permitem indicar **características específicas** de um componente.

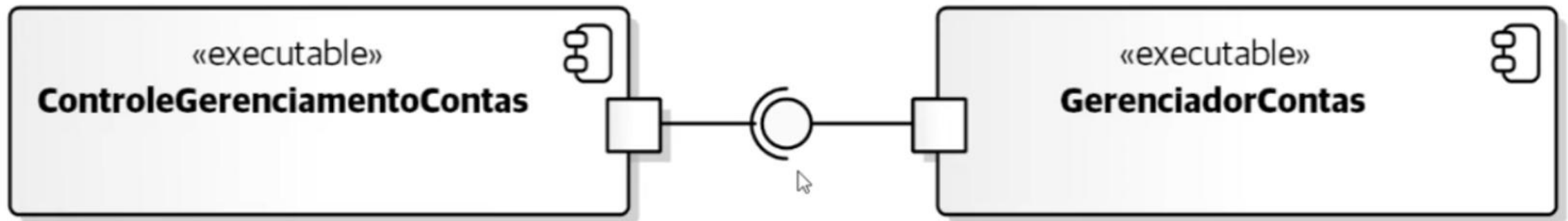


Interface

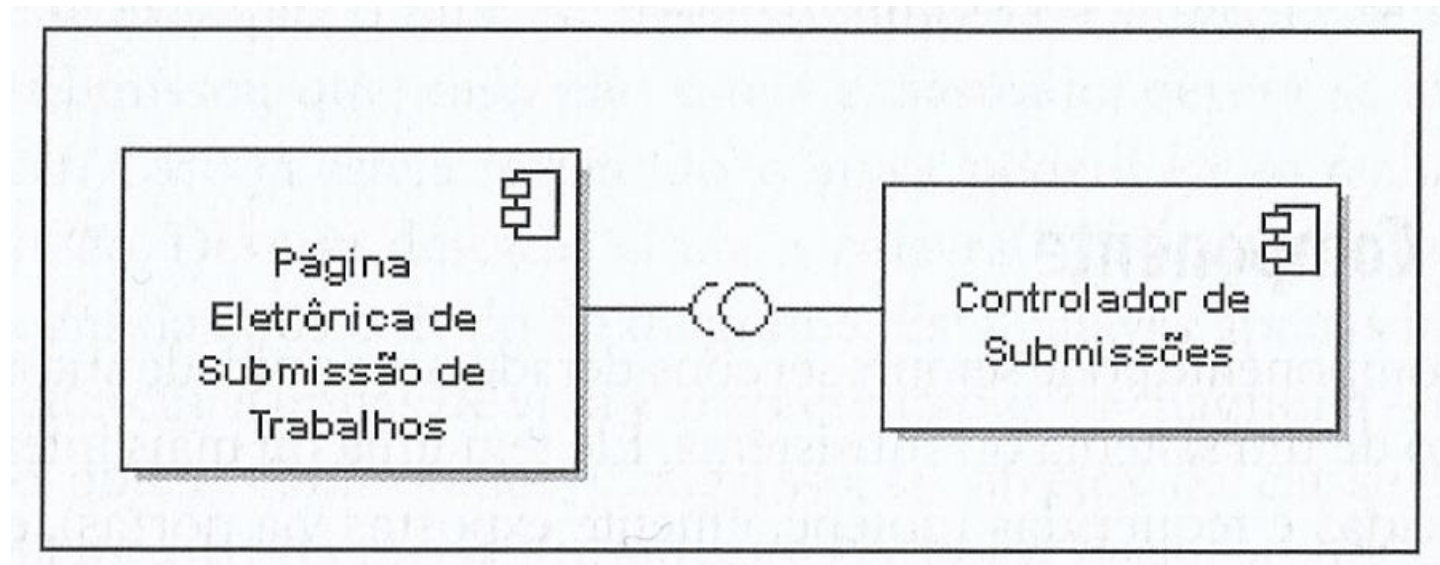
- Define os métodos que um componente disponibiliza para outros componentes. Pode ser **oferecida (provided)** ou **requerida (required)**.
- **Interfaces fornecidas** são representadas por um **círculo completo** na extremidade — esse símbolo (“pirulito”) é uma forma abreviada de representar uma **relação de realização** de uma interface.
- **Interfaces requeridas** são representadas por um **meio círculo** (também chamado de **soquete**) — indicam uma interface que o componente **necessita** para funcionar.



Interfaces Fornecidas e Requeridas

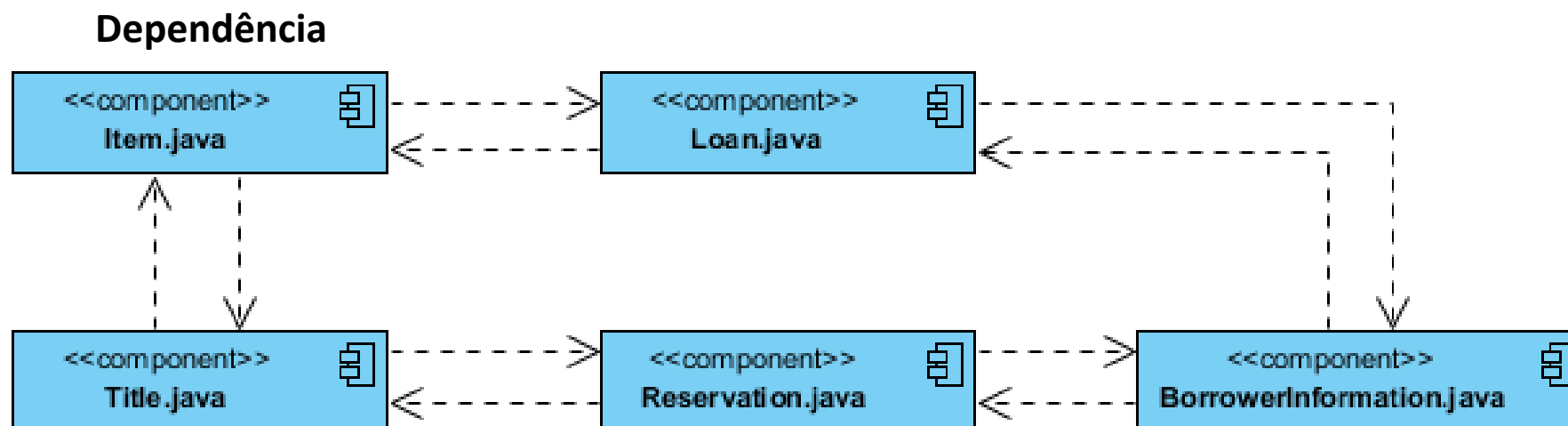


Interfaces Fornecidas e Requeridas



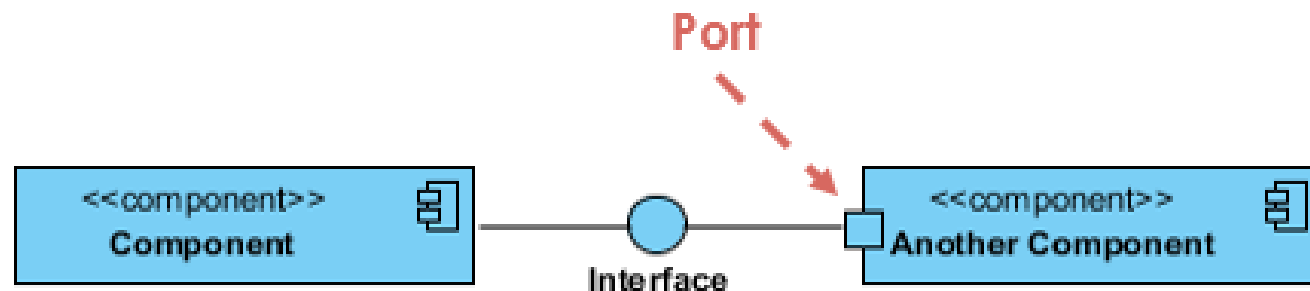
Dependência

- Uma **dependência** é um relacionamento que indica que um ou mais elementos do modelo **necessitam de outros elementos** para sua **especificação ou implementação**. É desenhada com uma **seta tracejada**.



Portas

- As **portas** são pontos de comunicação entre o interior do componente e o ambiente externo.
- Permitem que elementos internos **enviem ou recebam mensagens**;
- São frequentemente usadas **em componentes complexos**;
- Representadas graficamente como **quadrados** nas bordas do componente.

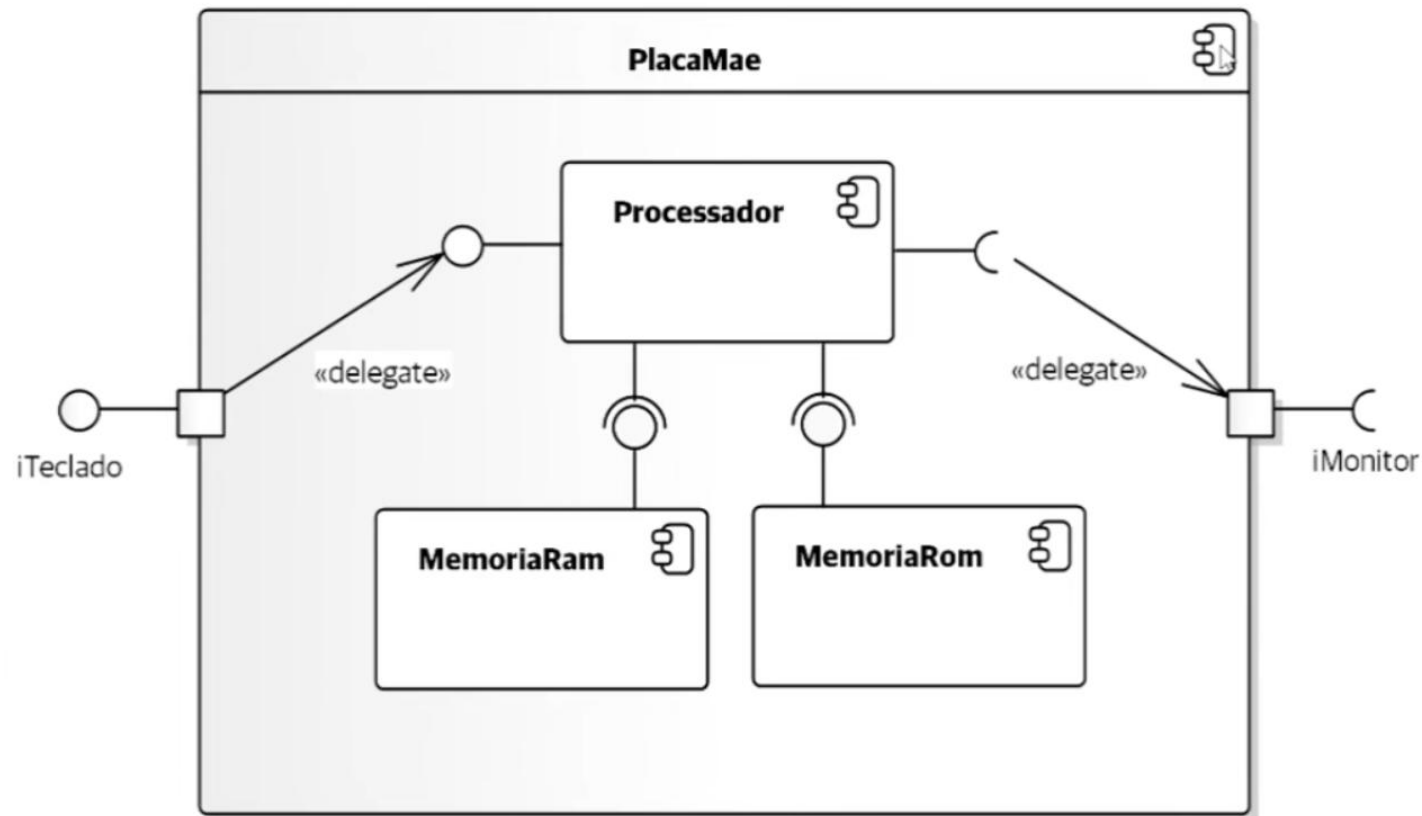


Classes e Componentes Internos

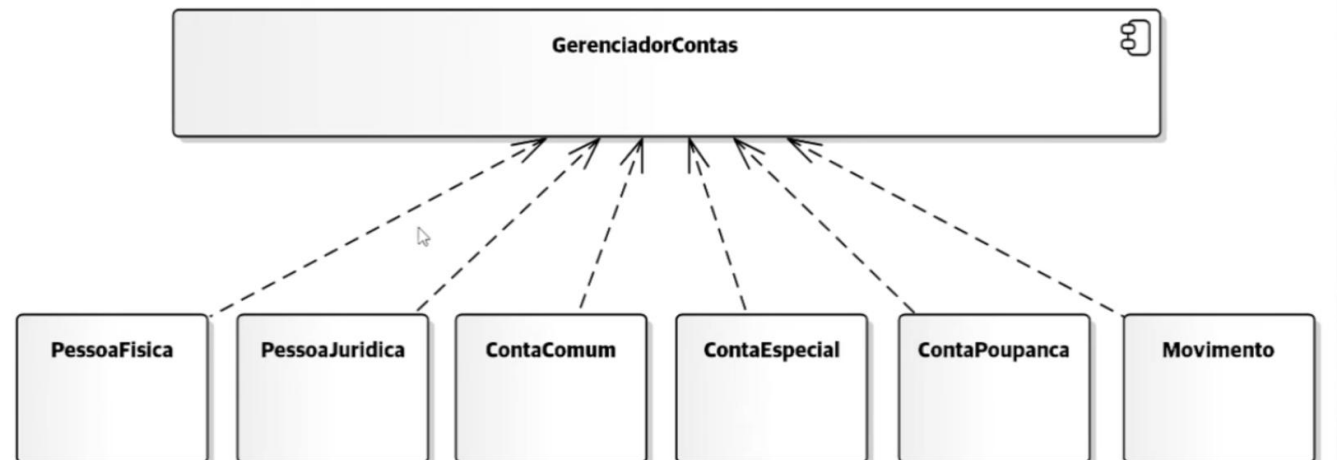
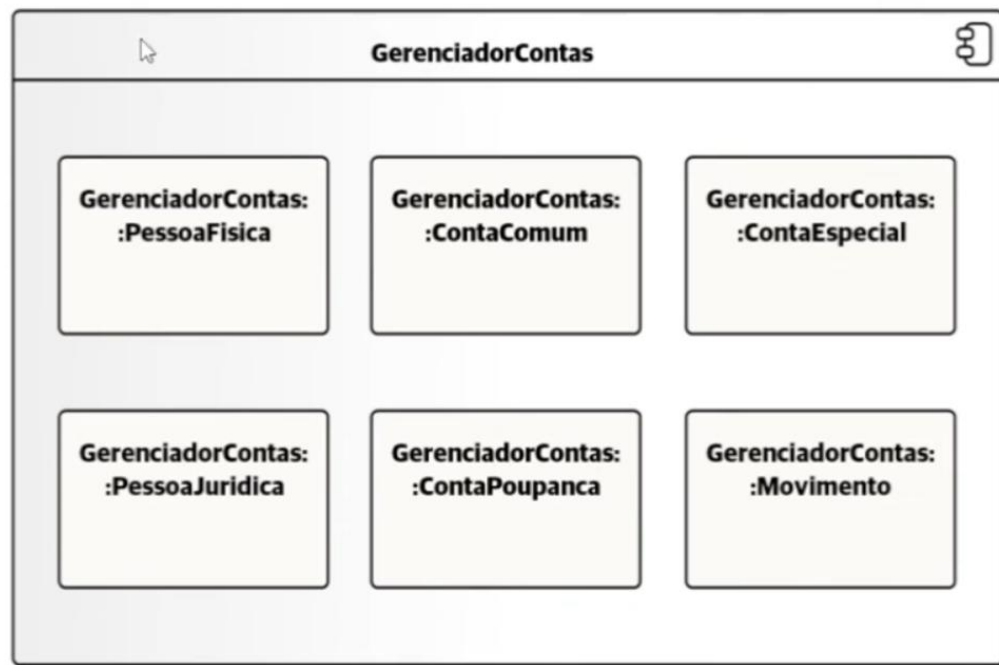
- Um componente pode **encapsular uma ou mais classes internas**.
- Existem duas visões possíveis:
 - **Caixa Preta (Black Box)**: Mostra apenas as interfaces do componente (oculta as classes internas).
 - **Caixa Branca (White Box)**: Mostra as classes internas e sua estrutura.



Visão da Caixa Branca de um Componente



Classes e Componentes Internos



Classes e Componentes Internos

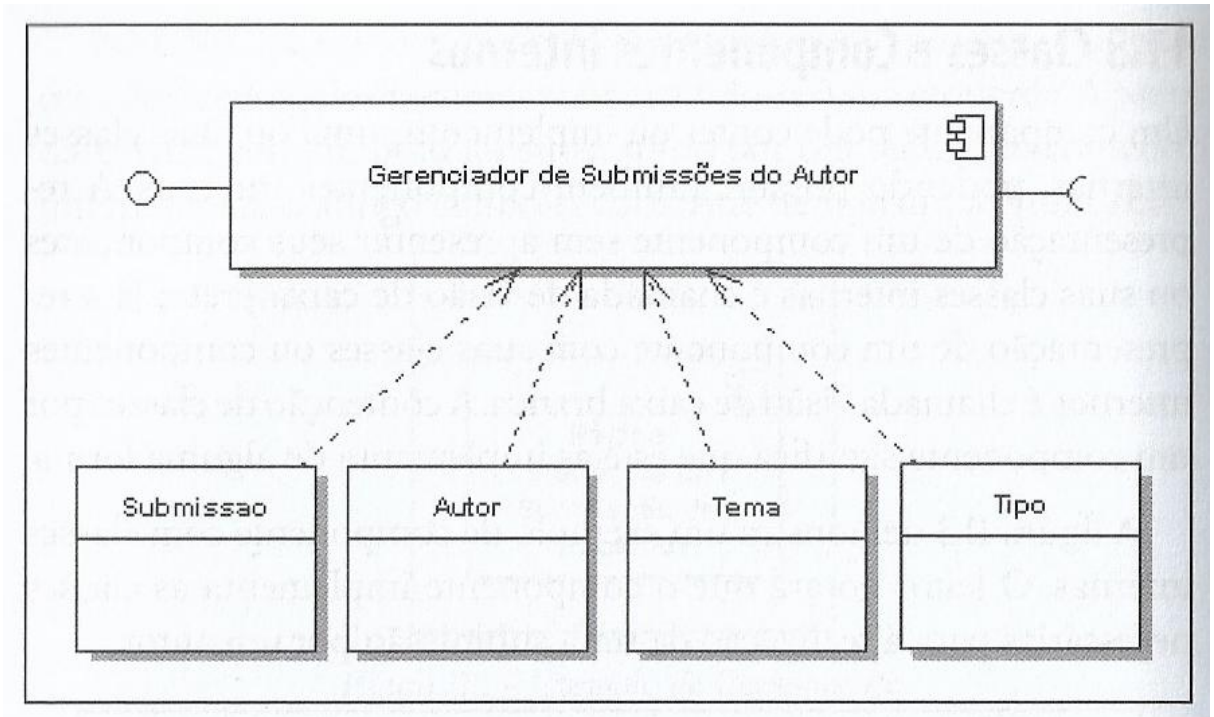
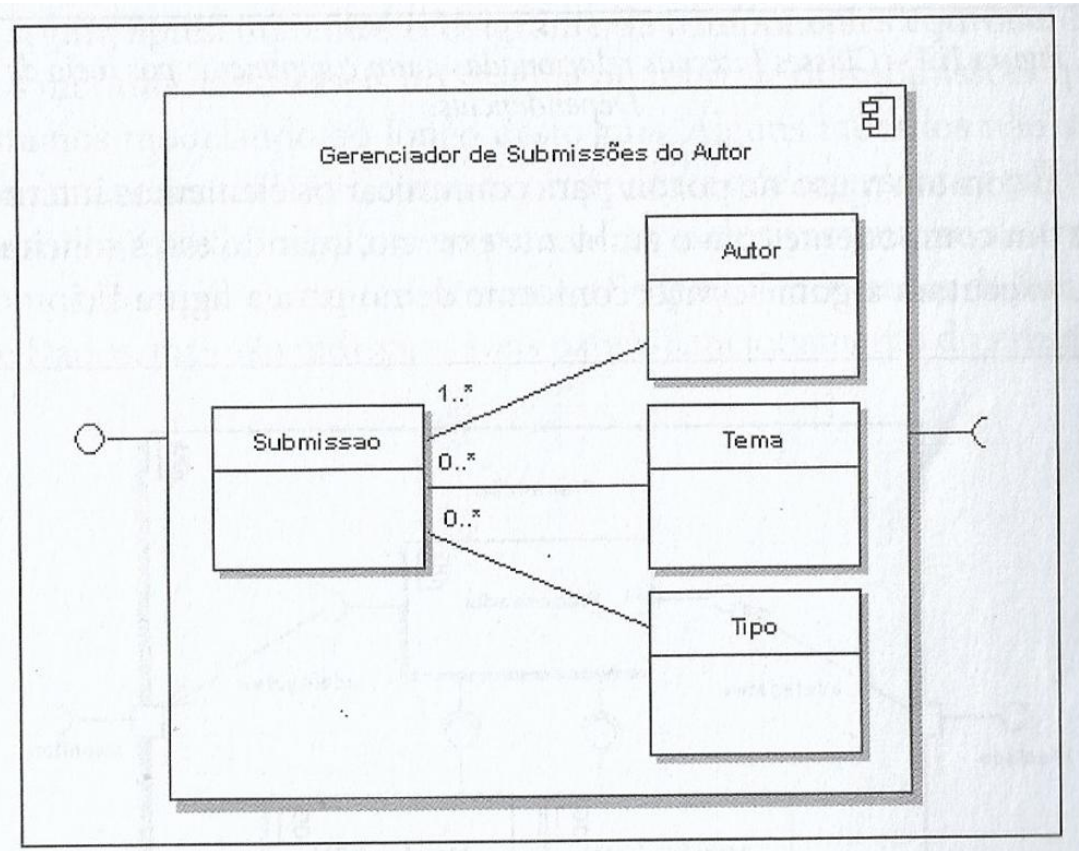


Diagrama do Sistema de Controle de Submissões

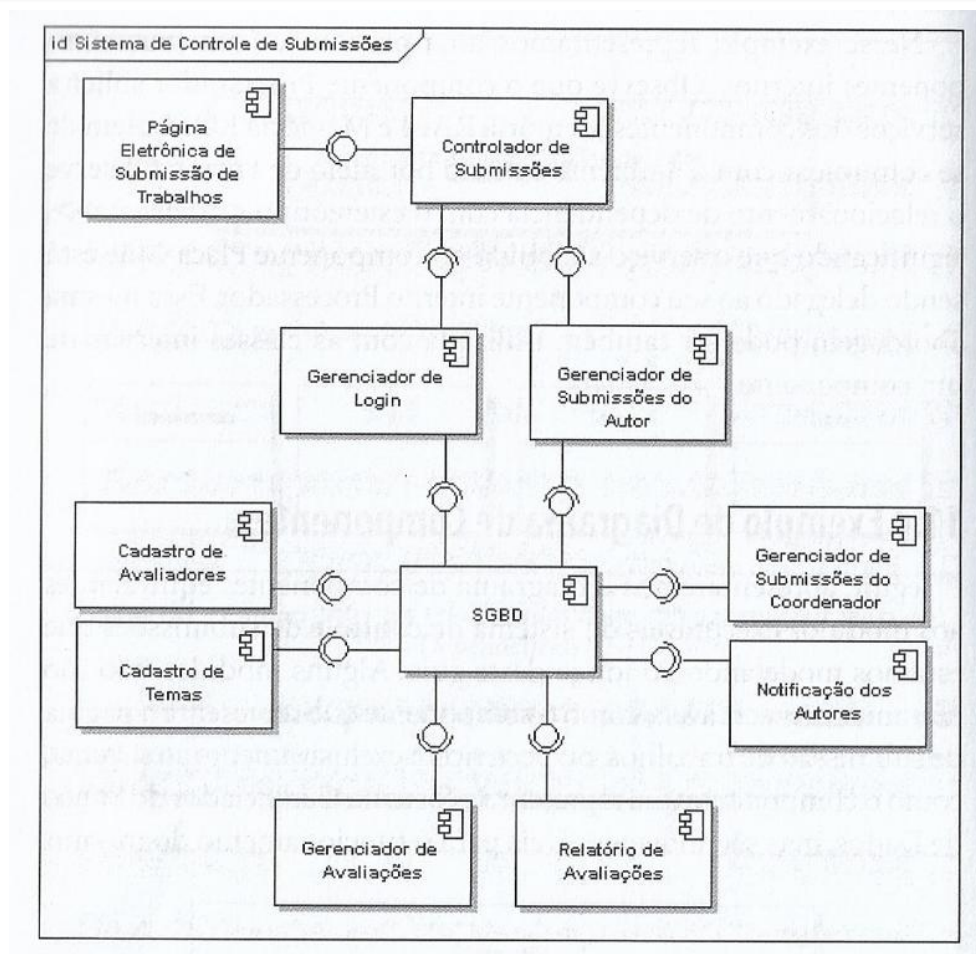
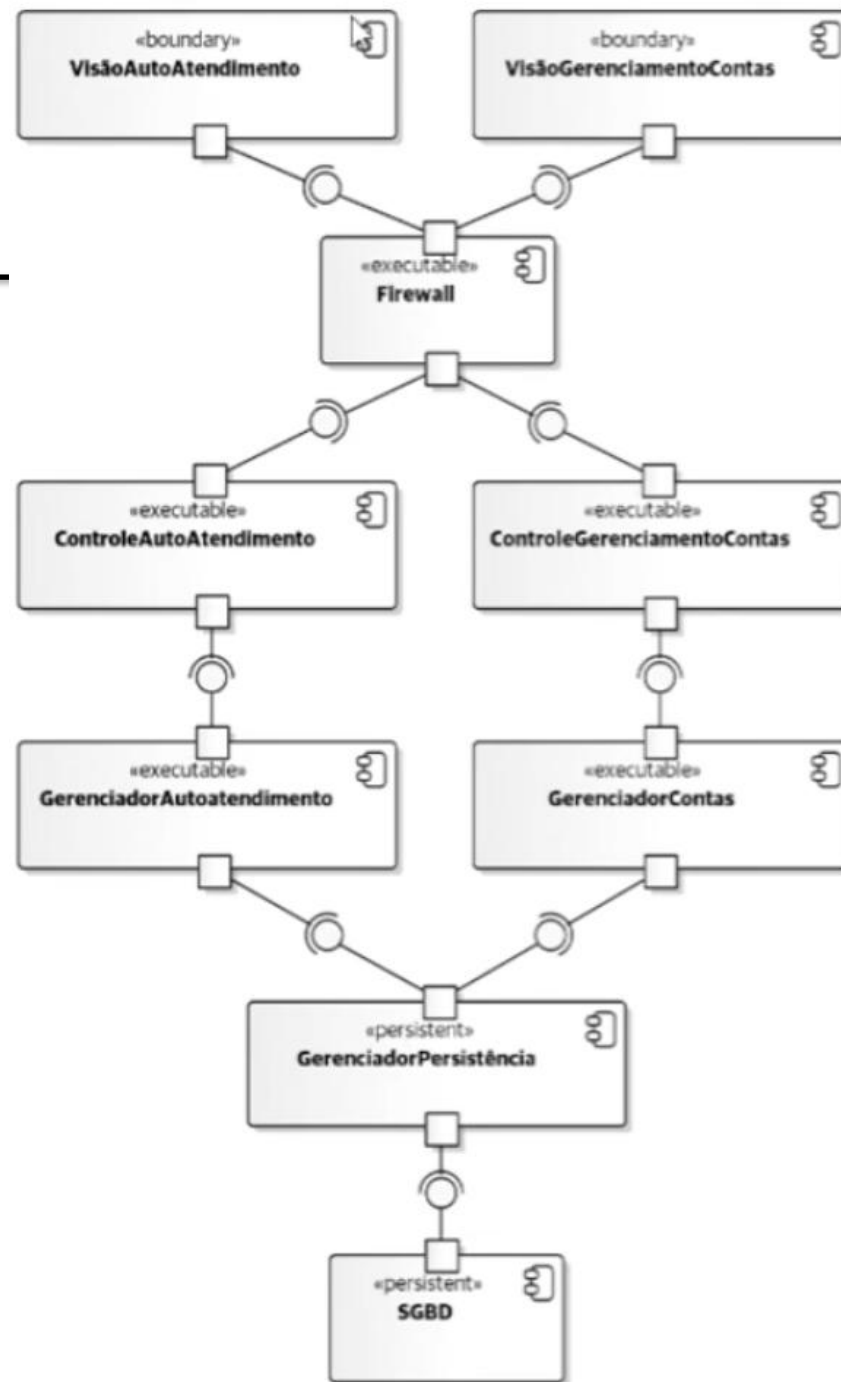
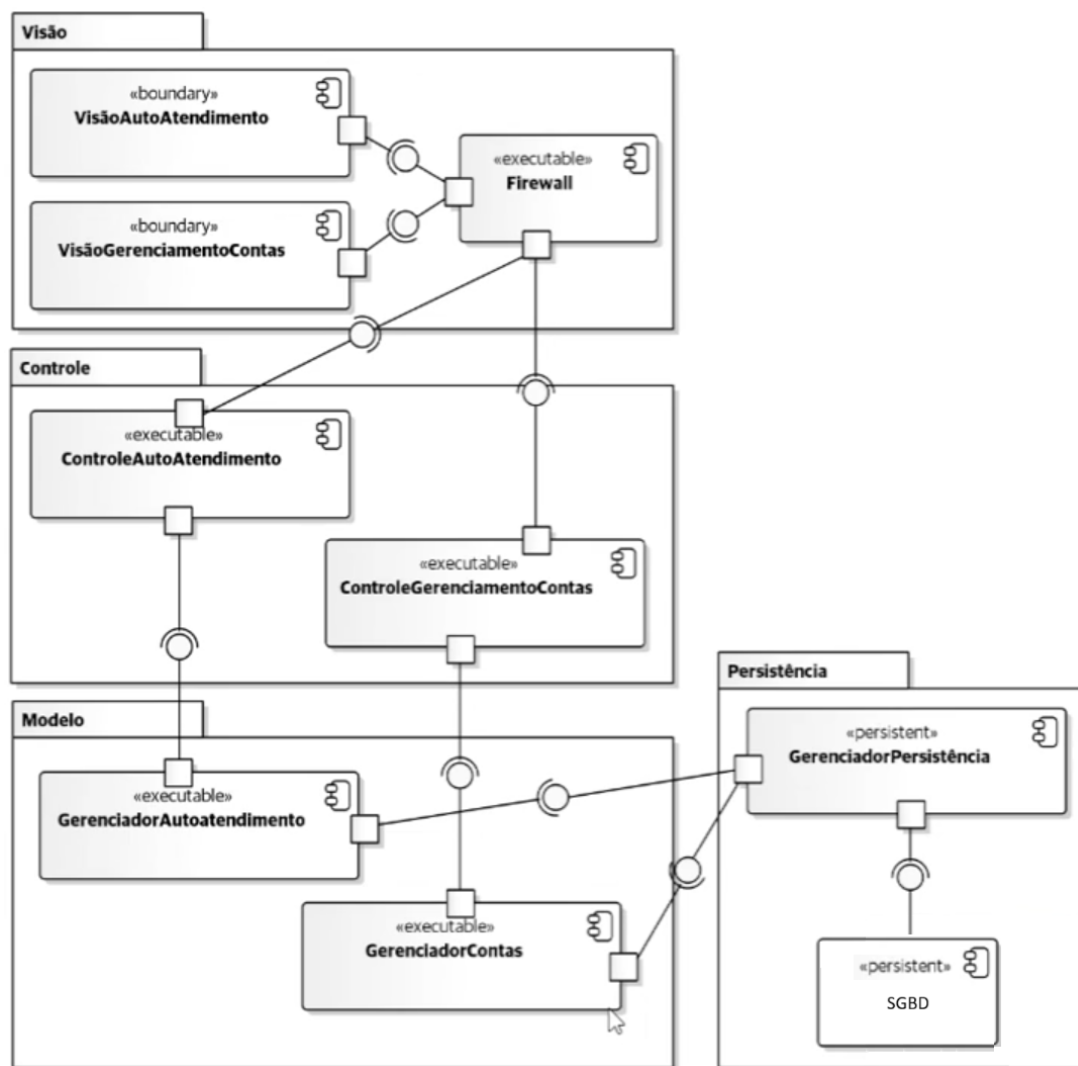


Diagrama de Componente e Diagrama de Pacotes



Referências

- BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. UML: guia do usuário. 2.ed., rev. e atual. Rio de Janeiro: Elsevier, 2005.
- FOWLER, Martin; SCOTT, Kendall. UML Essencial. São Paulo: Bookman, 2005.
- GUEDES, Gilleanes; UML 2 – Guia Prático - 2ª Edição. Editora Novatec, 2014.
- GUEDES, Gilleanes; Disponível em <https://youtu.be/0NnZHn0osp4>
- Visual Paradigm Learning Guides; Disponível em <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/>

